

PRODUCT DOCUMENTATION

Text Humanizer

User guide & technical documentation — measure the AI-style signals in any text, see them highlighted sentence by sentence, humanize the writing with a built-in engine, and prove the improvement with a before/after re-check.

Creator	**Dr. Asif Ahmad**
Product	Text Humanizer — version 1.1
Category	Writing
Published by	Asif SaaS Studio
Date	20 October 2026
Format	Single-file web application (index.html, 56.3 KB)

Private by design: the text never leaves the user's device — no uploads, no accounts, no tracking. And honest by design: the score is a stylistic estimate, never a promised detector verdict (Chapter 3).

Contents

1. What this product does
2. Getting text in — paste and document formats
3. The honest chapter — what the score is, and is not
4. The AI-style score — five measured signals
5. Colour highlighting — reading your text
6. The humanizing engine — tones and strengths
7. Reports — HTML, PDF, DOCX, DOC and JSON
8. How correctness was verified
9. Errors and what they mean
10. Limitations and honest scope
11. Privacy and security model
12. Frequently asked questions
13. Product information and credits
14. Appendix — the tested core, verbatim

1. What this product does

Text Humanizer answers two questions about any piece of writing: how machine-styled does this read? and can it be made to read as naturally human — measurably? You paste text or upload a document; the app scores it, highlights the machine-style patterns sentence by sentence, rewrites it with a built-in engine (four tones, three strengths), then scores the rewrite so the improvement is a number, not a claim. A full report can be downloaded in five formats.

Step	What happens
1 · Check	AI-style score (0–100%) from five measured signals, full metrics, colour-highlighted sentences.
2 · Humanize	Phrase-level and word-level rewriting, contraction restoration, sentence-rhythm repair.
3 · Re-check	The rewritten text is scored again; the point reduction is shown explicitly.
4 · Report	Download the complete before/after record as HTML, PDF, DOCX, DOC or JSON.

2. Getting text in — paste and document formats

Format	How it is read
Paste / type	Directly into the text box (50 to 20,000 characters).
.docx	A ZIP + XML reader inside the page extracts the paragraphs from Word's document.xml.
.odt	The same reader opens OpenDocument files and reads content.xml.
.pdf	A text extractor inflates the PDF's content streams (Flate, and ASCII85+Flate) and reads the text-showing operators. PDFs built from subset embedded fonts or scans do not yield readable text this way; the app detects that and asks for copy-paste instead of showing garbage.
.rtf	An RTF interpreter skips style/font destinations and decodes hex (\'93) and unicode (\u8212) escapes.
.html / .htm	Tags, scripts and styles are stripped; entities are decoded; block structure becomes paragraphs.
.txt / .md	Read as plain text.
.doc (legacy binary)	Detected and refused with guidance to Save As .docx — the OLE piece-table format cannot be read reliably in a web page, and the app says so rather than guessing.

Files up to 4 MB are accepted; extracted text longer than 20,000 characters is loaded as its first 20,000 with a clear notice.

3. The honest chapter — what the score is, and is not

> **The score in this product is a stylistic estimate, computed on the device. It is not a GPTZero, Turnitin, Originality.ai, Copyleaks or Pangram score, and this product does not claim to pass those systems.**

Those detectors are proprietary, server-side models, retrained continuously; their verdicts on the same text differ from one another and change over time. No static web page can replicate them, and any tool that guarantees a pass is making a marketing claim, not an engineering one. What this product does is real and verifiable: it measures the documented stylistic patterns such systems are known to weigh — stock machine phrasing, monotonous sentence rhythm, formal-word density, missing contractions, flat vocabulary — reduces them, and shows the measured change. Used properly, it is a writing tool: run it on your own drafts, review every rewrite yourself, and keep your own voice in charge.

If genuine vendor scores are ever required, the route is a small backend holding paid API keys (GPTZero and Originality.ai sell API access; Turnitin offers no public self-serve API)

that the page calls over HTTPS — a different product shape, with per-scan costs and text leaving the device. Until that exists, the local estimator is the honest deliverable.

4. The AI-style score — five measured signals

Signal	Weight	What it measures
Stock phrases	30%	Hits against a 52-pattern library of machine-typical phrasing ("it is important to note that", "furthermore", "delve into", "plays a crucial role"…), per 100 words.
Sentence rhythm	24%	The coefficient of variation of sentence lengths. Machine prose is metronomic (low variation); human prose bursts (high variation).
Formal-word density	18%	Occurrences of a 90-word formal/AI-favoured lexicon (utilize, leverage, facilitate, numerous…) per 100 words.
Vocabulary diversity	18%	Type–token ratio compared with the expectation for the text's length.
Contraction absence	10%	Prose of 60+ words with no contractions at all is a recognised machine tell.

Bands: **0–31** mostly human-style · **32–59** mixed · **60–100** strong AI-style signals. Texts under 50 words are scored at reduced confidence — style statistics need material.

5. Colour highlighting — reading your text

Every sentence is classified and painted: **red** — strong AI-style signals (stock phrase, or several signals together); **amber** — some signals; **green** — human-style. A sentence earns points for containing a stock phrase, for heavy formal-word use, for sitting exactly on the mean sentence length, and for length without a single contraction. The highlighted view is included in the HTML, DOCX and DOC reports, so the evidence travels with the report.

6. The humanizing engine — tones and strengths

Stage	What it does
Phrase rules (all strengths)	34 rules replace or remove stock phrasing, rotating natural alternatives ("Furthermore," → "Also," / "On top of that," / "And") so the rewrite does not develop a new monotone.
Word swaps (Medium+)	~90 formal words swapped for plain equivalents (utilize → use, obtain → get), keeping most occurrences so the voice shifts without flattening.
Contractions (Medium+)	"do not" → "don't", "it is" → "it's" — skipped in Academic tone, where formality is the point.
Rhythm repair (Strong)	Very long sentences are split at safe conjunctions; runs of very short sentences are merged — directly attacking the uniformity signal.

Tones: Natural (everyday writing) · Academic (scholarly, no forced contractions) · Casual (friendliest, contractions throughout) · Professional (clear business register). **Strengths:** Light (phrases only), Medium (default), Strong (adds rhythm repair). The engine is deterministic — the same input always yields the same rewrite — and it never invents facts: it changes wording and rhythm, not content. Numbers, names and claims pass through untouched, which is why a human review of the result still matters.

7. Reports — HTML, PDF, DOCX, DOC and JSON

Format	Contents / how it is produced
HTML	A styled report page: both scores, both metric tables, both highlighted texts.
PDF	Produced by a real text-PDF writer inside the page (paginated, wrapped, valid xref) — scores, metrics, and both texts.
DOCX	A genuine Word file assembled by a hand-written ZIP writer (stored entries + CRC32), with the sentence highlighting carried as run shading.
DOC	The HTML report delivered as a Word-compatible .doc (Word opens it as an editable document) — the legacy binary format cannot be authored in a page, and the report says so.
JSON	The complete structured record: metadata, both analyses (scores, metrics, per-sentence classes), and both full texts — for pipelines and archives.

8. How correctness was verified

8.1 Calibration

A deliberately machine-styled sample scores **62%** (14 stock phrases flagged); a genuinely hand-written sample scores **0%**. After humanizing (Natural, Medium) the machine sample falls to **28%** — stock phrases 14 → 2, contractions restored, length preserved. 16/16 engine checks pass, including abbreviation-safe sentence splitting (Dr., e.g., i.e.) and deterministic output.

8.2 Format parsers, against real files

DOCX and ODT fixtures in the real (deflated ZIP) formats extract exactly; RTF decodes quotes and unicode without leaking its font table; HTML strips scripts/styles; a ReportLab PDF (ASCII85+Flate streams) extracts word-perfect, while a subset-font PDF produces the designed clean error. Two genuine bugs were caught and fixed in testing: a stray newline corrupting PDF stream inflation, and the stream scanner matching inside the word "endstream".

8.3 Writers, against real readers

The CRC32 matches the standard check vector; the ZIP output validates in Python's zipfile; an app-exported DOCX opens in python-docx with every section present; an app-exported PDF is read back by poppler with its text intact.

9. Errors and what they mean

Message (short form)	Cause and remedy
Enter at least 50 characters	Style statistics need a sentence or two of material.
Text longer than 20,000 characters	Trim, or split the document and process it in parts.
Legacy .doc cannot be read	Open in Word → Save As .docx or .txt → upload that.
Readable text could not be extracted from this PDF	Scanned or subset-font PDF — copy the text from your PDF reader and paste it.
File larger than 4 MB	Use a smaller document or paste an excerpt.

10. Limitations and honest scope

- The score is an estimator of style, not authorship: a careful human writing formally can score amber; the tool measures patterns, not provenance.

- The rewriter is rule-based; it cannot judge meaning the way an editor can. Always review the output — especially quotations, technical terms and legal or medical wording.
- No OCR: scanned documents must be pasted as text.
- Detector scores from commercial systems are neither reproduced nor promised (Chapter 3).

11. Privacy and security model

- Everything is local — parsing, scoring and rewriting run in the page; the analysed text is never uploaded anywhere.
- No localStorage, cookies, accounts, analytics or network calls of any kind; the page works fully offline.
- No service workers, permission prompts or parent-window access — sandbox-iframe safe.

12. Frequently asked questions

Will this make my text "pass" Turnitin or GPTZero? No tool can promise that, including this one — see Chapter 3. It measurably removes the style patterns those systems weigh; the final judgement, and the final responsibility, stays with the author.

Why did my own writing score amber? Formal registers genuinely share signals with machine prose (few contractions, even rhythm). The highlighting shows exactly which sentences and why — treat it as editing feedback, not an accusation.

Does it work on documents, or only pasted text? Both — Word, PDF, RTF, HTML, ODT and plain text uploads are read in the page (Chapter 2).

Is the rewrite safe to use as-is? Read it first. The engine preserves content but it is a machine editor; your review is the quality step.

13. Product information and credits

Field	Detail
Product	Text Humanizer
Version	1.1 (20 October 2026) — v1.1 added document intake and report exports
Creator	**Dr. Asif Ahmad**
Publisher	Asif SaaS Studio
Category / slug	Writing / text-humanizer
Deliverable	index.html — 56.3 KB, single file, all CSS and JavaScript inline
Requirements	Any modern browser; no server, no installation, no account
Verification	16/16 engine checks; calibration 62% → 28%; parsers and writers verified against real files and readers (Chapter 8)

Appendix — the tested core, verbatim

The code below is the complete working core of the product exactly as it ships inside `index.html`: the lexicons, the five-signal analysis, the humanizing engine, and the format engines (ZIP/Office readers, RTF and HTML parsers, PDF text extraction, CRC32 + ZIP writer, and the text-PDF writer). Interface wiring is omitted.

```
/* Stock phrases strongly associated with machine-generated prose */
var STOCK_PHRASES = [
  "it is important to note that", "it is worth noting that", "it should be noted that",
  "in today's fast-paced world", "in the ever-evolving", "in conclusion", "furthermore",
  "moreover", "additionally", "in addition", "overall", "in summary", "to summarize",
  "delve into", "delves into", "when it comes to", "plays a crucial role", "plays a vital role",
  "plays an important role", "a wide range of", "a variety of", "in order to", "due to the fact that",
  "at the end of the day", "on the other hand", "in terms of", "as a result", "consequently",
  "nevertheless", "nonetheless", "it goes without saying", "needless to say", "first and foremost",
  "last but not least", "in the realm of", "the world of", "landscape of", "ever-changing landscape",
  "unlock the power", "unlock the potential", "harness the power", "game-changer", "game changer",
  "cutting-edge", "state-of-the-art", "seamlessly", "furthermore,", "in this article, we will",
  "in this essay", "has revolutionized", "has transformed the way", "stands as a testament",
  "a testament to", "rich tapestry", "vibrant tapestry", "intricate tapestry"
];
/* Formal / AI-favoured words -> plainer human alternatives */
var WORD_SWAPS = {
  utilize: ["use", "work with"], utilise: ["use"], leverage: ["use", "build on"],
  facilitate: ["help", "make easier"], numerous: ["many", "plenty of"], sufficient: ["enough"],
  approximately: ["about", "around"], demonstrate: ["show"], indicate: ["show", "suggest"],
  therefore: ["so"], thus: ["so"], hence: ["so"], purchase: ["buy"], assistance: ["help"],
  assist: ["help"], commence: ["start", "begin"], terminate: ["end", "stop"],
  endeavor: ["try", "effort"], endeavour: ["try", "effort"], attempt: ["try"],
  obtain: ["get"], receive: ["get"], require: ["need"], additional: ["more", "extra"],
  furthermore: ["also"], moreover: ["also", "and"], additionally: ["also"],
  consequently: ["so", "as a result"], nevertheless: ["still", "yet"], nonetheless: ["still", "yet"],
  subsequently: ["later", "after that"], previously: ["before", "earlier"],
  currently: ["now"], initially: ["at first"], ultimately: ["in the end", "finally"],
  essential: ["key", "vital"], crucial: ["key", "critical"], vital: ["key", "essential"],
  significant: ["major", "big"], substantially: ["greatly", "a lot"], primarily: ["mainly"],
  effectively: ["in practice"], rapidly: ["quickly"],
  individually: ["one by one"], components: ["parts"], comprehensive: ["full", "complete"],
```

```
  incorporate: ["include", "build in"], implement: ["put in place", "carry out"],
  ensure: ["make sure"], enhance: ["improve", "strengthen"], improve: ["make better"],
  increase: ["raise", "grow"], decrease: ["lower", "drop"], determine: ["work out", "find"],
  establish: ["set up"], regarding: ["about", "on"], concerning: ["about"],
  prior: ["before"], therefore_: [], individuals: ["people"], persons: ["people"],
  children_: [], "in order to": ["to"], "due to the fact that": ["because"],
  "a large number of": ["many"], "a wide range of": ["many kinds of", "all sorts of"],
  "is able to": ["can"], "are able to": ["can"], "has the ability to": ["can"],
  "in the event that": ["if"], "at this point in time": ["now"], "in spite of the fact that": ["although"],
  methodology: ["method"], utilize_: []
};
Object.keys(WORD_SWAPS).forEach(function(k){ if (!WORD_SWAPS[k].length || k.endsWith("_")) delete
WORD_SWAPS[k]; });
var CONTRACTIONS = [
  [/\bdo not\b/gi, "don't"], [/\bdoes not\b/gi, "doesn't"], [/\bdid not\b/gi, "didn't"],
  [/\bcannot\b/gi, "can't"], [/\bcan not\b/gi, "can't"], [/\bwill not\b/gi, "won't"],
  [/\bwould not\b/gi, "wouldn't"], [/\bcould not\b/gi, "couldn't"], [/\bshould not\b/gi, "shouldn't"],
  [/\bis not\b/gi, "isn't"], [/\bare not\b/gi, "aren't"], [/\bwas not\b/gi, "wasn't"],
  [/\bwere not\b/gi, "weren't"], [/\bhave not\b/gi, "haven't"], [/\bhas not\b/gi, "hasn't"],
  [/\bit is\b/gi, "it's"], [/\bthat is\b/gi, "that's"], [/\bthere is\b/gi, "there's"],
  [/\bwe are\b/gi, "we're"], [/\byou are\b/gi, "you're"], [/\bthey are\b/gi, "they're"],
  [/\bi am\b/gi, "I'm"], [/\bwe will\b/gi, "we'll"], [/\bit will\b/gi, "it'll"],
  [/\bdo not\b/gi, "don't"]
];
/* Phrase-level rewrite rules: [regex, [alternatives]] — "" alternative = delete */
var PHRASE_RULES = [
  [/\bit is important to note that\s*/gi, ["", "keep in mind: ", "note that "]],
  [/\bit is worth noting that\s*/gi, ["", "note that "]],
  [/\bit should be noted that\s*/gi, ["", "note that "]],
  [/\bin today's fast-paced world,?\s*/gi, ["these days, ", "nowadays, ", ""]],
  [/\bin the ever-evolving world of\s*/gi, ["in ", "in today's "]],
  [/\bin conclusion,?\s*/gi, ["in short, ", "so, ", "to wrap up, "]],
  [/\bfurthermore,?\s*/gi, ["also, ", "on top of that, ", "and "]],
  [/\bmoreover,?\s*/gi, ["also, ", "what's more, ", "and "]],
  [/\badditionally,?\s*/gi, ["also, ", "plus, ", ""]],
  [/\bin addition,?\s*/gi, ["also, ", "plus, "]],
  [/\bdelve into\s*/gi, ["look at ", "dig into ", "explore "]],
  [/\bdelves into\s*/gi, ["looks at ", "explores "]],
  [/\bwhen it comes to\s*/gi, ["for ", "with "]],
  [/\bplays a crucial role in\s*/gi, ["is central to ", "matters a lot for "]],
  [/\bplays a vital role in\s*/gi, ["is central to ", "matters a lot for "]],
  [/\bplays an important role in\s*/gi, ["matters for ", "is a big part of "]],
  [/\bdue to the fact that\s*/gi, ["because "]],
  [/\bin order to\s*/gi, ["to "]],
  [/\bat the end of the day,?\s*/gi, ["in the end, ", "ultimately, ", ""]],
  [/\bit goes without saying that\s*/gi, ["clearly, ", ""]],
  [/\bneedless to say,?\s*/gi, ["of course, ", ""]],
  [/\bfirst and foremost,?\s*/gi, ["first, ", "above all, "]],
  [/\blast but not least,?\s*/gi, ["finally, ", "and last, "]],
  [/\bstands as a testament to\s*/gi, ["shows ", "proves "]],
  [/\ba testament to\s*/gi, ["proof of ", "a sign of "]],
  [/\bhas revolutionized\s*/gi, ["has changed ", "has reshaped "]],
  [/\bhas transformed the way\s*/gi, ["has changed how "]],
  [/\bseamlessly\s*/gi, ["smoothly ", "without friction "]],
  [/\bcutting-edge\s*/gi, ["latest ", "newest "]],
  [/\bstate-of-the-art\s*/gi, ["latest ", "most advanced "]],
  [/\bgame-changer\b/gi, ["turning point", "big shift"]],
  [/\bunlock the potential of\s*/gi, ["make the most of ", "get more from "]],
  [/\bharness the power of\s*/gi, ["use ", "put to work "]],
  [/\ba wide range of\s*/gi, ["all kinds of ", "many types of "]],
  [/\bin the realm of\s*/gi, ["in ", "in the world of "]]
];
/* ---------- text utilities ---------- */
function splitParagraphs(text){ return String(text).split(/\n\s*\n|\n/).map(function(p){ return
p.trim(); }).filter(Boolean); }
function splitSentences(text){
  var guarded = String(text)
    .replace(/\b(Dr|Mr|Mrs|Ms|St|Prof|Sr|Jr|vs|etc|approx|Fig|No)\./gi, "$1\u0001")
    .replace(/\be\.g\./gi, "e\u0001g\u0001")
    .replace(/\bi\.e\./gi, "i\u0001e\u0001");
  var parts = guarded.match(/[^.!?]+[.!?]+["']?\s*|[^.!?]+$/g) || [guarded];
  return parts.map(function(s){ return s.replace(/\u0001/g, ".").trim(); }).filter(Boolean);
}
function wordsOf(text){ return String(text).toLowerCase().match(/[a-z0-9']+/g) || []; }
function countSyllables(w){
  w = w.toLowerCase().replace(/[^a-z]/g, "");
  if (!w) return 0; if (w.length <= 3) return 1;
```

```
  var s = (w.replace(/e$/, "").match(/[aeiouy]+/g) || []).length;
  return Math.max(1, s);
}
function flesch(text, words, sentences){
  if (!words.length || !sentences.length) return 0;
  var syl = words.reduce(function(a, w){ return a + countSyllables(w); }, 0);
  return Math.max(0, Math.min(100, 206.835 - 1.015 * (words.length / sentences.length) - 84.6 * (syl /
words.length)));
}
function occurrences(haystackLower, phrase){ var n = 0, i = 0; while ((i = haystackLower.indexOf(phrase, i)) >=
0){ n++; i += phrase.length; } return n; }
/* ---------- analysis ---------- */
function sentenceClass(sentence, lower, meanLen){
  var pts = 0, wc = wordsOf(sentence).length;
  STOCK_PHRASES.forEach(function(p){ if (lower.indexOf(p) >= 0) pts += 2; });
  var formal = 0;
  wordsOf(sentence).forEach(function(w){ if (WORD_SWAPS[w]) formal++; });
  if (formal >= 2) pts += 1;
  if (!/('[a-z]+\b|n't\b)/i.test(sentence) && wc >= 15) pts += 0.5;
  if (meanLen && Math.abs(wc - meanLen) <= meanLen * 0.15 && wc >= 10) pts += 0.5;
  return pts >= 2 ? "high" : pts >= 1 ? "med" : "low";
}
function analyze(text){
  var clean = String(text || "").trim();
  if (!clean) return { score: 0, words: 0, sentences: 0, stockHits: 0, stockPer100: 0, avgSentenceLen: 0,
    sentenceCV: 0, formalPer100: 0, contractions: 0, ttr: 0, flesch: 0, sentenceObjs: [] };
  var words = wordsOf(clean), lower = clean.toLowerCase();
  var paras = splitParagraphs(clean);
  var sentences = [];
  paras.forEach(function(p){ splitSentences(p).forEach(function(s){ sentences.push(s); }); });
  var stockHits = 0;
  STOCK_PHRASES.forEach(function(p){ stockHits += occurrences(lower, p); });
  var stockPer100 = words.length ? 100 * stockHits / words.length : 0;
  var lens = sentences.map(function(s){ return wordsOf(s).length; }).filter(function(n){ return n > 0; });
  var avgLen = lens.length ? lens.reduce(function(a, b){ return a + b; }, 0) / lens.length : 0;
  var sdLen = lens.length > 1 ? Math.sqrt(lens.reduce(function(a, l){ return a + (l - avgLen) * (l - avgLen); },
0) / (lens.length - 1)) : 0;
  var cv = avgLen ? sdLen / avgLen : 0;
  var formalHits = 0;
  words.forEach(function(w){ if (WORD_SWAPS[w]) formalHits++; });
  var formalPer100 = words.length ? 100 * formalHits / words.length : 0;
  var contractions = (clean.match(/\b\w+n't\b|\b\w+'(re|ve|ll|d|s)\b/gi) || []).length;
  var unique = {};
  words.forEach(function(w){ unique[w] = 1; });
  var ttr = words.length ? Object.keys(unique).length / words.length : 0;
  var expTTR = words.length < 100 ? 0.82 : words.length < 300 ? 0.70 : words.length < 800 ? 0.60 : words.length
< 2000 ? 0.51 : 0.43;
  var sub = {
    stock: Math.min(1, stockPer100 / 5),
    uniformity: lens.length < 3 ? 0.35 : Math.max(0, Math.min(1, 1 - cv / 0.55)),
    formal: Math.min(1, formalPer100 / 4),
    contraction: words.length >= 60 ? Math.max(0, Math.min(1, 1 - (100 * contractions / words.length) / 1.1)) :
0.3,
    diversity: Math.max(0, Math.min(1, (expTTR - ttr + 0.05) / 0.22))
  };
  var score = Math.round(100 * (0.30 * sub.stock + 0.24 * sub.uniformity + 0.18 * sub.formal + 0.10 *
sub.contraction + 0.18 * sub.diversity));
  if (words.length < 50) score = Math.round(score * 0.7); // too little text = low confidence
  return {
    score: Math.max(0, Math.min(100, score)),
    words: words.length, sentences: sentences.length,
    stockHits: stockHits, stockPer100: stockPer100,
    avgSentenceLen: avgLen, sentenceCV: cv,
    formalPer100: formalPer100, contractions: contractions,
    ttr: ttr, flesch: flesch(clean, words, sentences),
    sentenceObjs: sentences.map(function(s){ return { text: s, cls: sentenceClass(s, s.toLowerCase(),
avgLen) }; })
  };
}
function bandOf(score){
  return score >= 60 ? ["Strong AI-style signals", "#B3261E"] :
    score >= 32 ? ["Mixed — AI-style patterns present", "#8A5A00"] :
    ["Mostly human-style", "#146C43"];
}
/* ---------- humanizer ---------- */
function matchCase(replacement, matched){
  if (!replacement) return "";
```

```
  return /^[A-Z]/.test(matched) ? replacement.charAt(0).toUpperCase() + replacement.slice(1) : replacement;
}
function humanize(text, tone, strength){
  var rot = 0;
  function pick(alts){ return alts[(rot++) % alts.length]; }
  var strong = strength === "strong", medium = strength !== "light";
  var paras = splitParagraphs(text).map(function(para){
    var out = para;
    // 1) phrase rules (all strengths)
    PHRASE_RULES.forEach(function(rule){
      out = out.replace(rule[0], function(m){ return matchCase(pick(rule[1]), m); });
    });
    // 2) word swaps (medium+)
    if (medium){
      Object.keys(WORD_SWAPS).forEach(function(w){
        if (w.indexOf(" ") >= 0) return; // multi-word handled by phrase rules
        var re = new RegExp("\\b" + w + "\\b", "gi");
        out = out.replace(re, function(m, offset, whole){
          // deterministic variety: swap most occurrences, keep some originals at medium
          var h = (offset * 31 + whole.length) % 10;
          if (!strong && h < 3) return m;
          return matchCase(pick(WORD_SWAPS[w]), m);
        });
      });
      // multi-word word swaps ("in order to" style already covered; is able to etc.)
      [["is able to", "can"], ["are able to", "can"], ["has the ability to", "can"],
       ["in the event that", "if"], ["at this point in time", "now"], ["a large number of", "many"]
      ].forEach(function(pair){
        out = out.replace(new RegExp("\\b" + pair[0] + "\\b", "gi"), function(m){ return matchCase(pair[1],
m); });
      });
    }
    // 3) contractions (medium+; academic keeps prose more formal — fewer)
    if (medium && tone !== "academic"){
      CONTRACTIONS.forEach(function(pair){
        out = out.replace(pair[0], function(m){
          if (!strong && (m.length * 7) % 3 === 0) return m;
          return matchCase(pair[1], m);
        });
      });
    }
    // 4) rhythm (strong): split very long sentences; merge very short neighbours
    if (strong){
      var sents = splitSentences(out), rebuilt = [];
      sents.forEach(function(s){
        var wc = wordsOf(s).length;
        if (wc > 27){
          var splitAt = s.search(/,\s+(and|but|while|which)\s+/i);
          if (splitAt > 30){
            var head = s.slice(0, splitAt).trim(), tail = s.slice(splitAt + 1).trim();
            tail = tail.replace(/^(and|but)\s+/i, "");
            rebuilt.push(head + ".");
            rebuilt.push(tail.charAt(0).toUpperCase() + tail.slice(1));
            return;
          }
        }
        rebuilt.push(s);
      });
      var merged = [];
      for (var i = 0; i < rebuilt.length; i++){
        var cur = rebuilt[i];
        if (merged.length && wordsOf(merged[merged.length - 1]).length < 7 && wordsOf(cur).length < 12 &&
            /^(It|This|They|He|She|We)\b/.test(cur)){
          merged[merged.length - 1] = merged[merged.length - 1].replace(/[.!?]\s*$/, "") + ", and " +
            cur.charAt(0).toLowerCase() + cur.slice(1);
        } else merged.push(cur);
      }
      out = merged.join(" ");
    }
    return out;
  });
  var joined = paras.join("\n\n").replace(/\s+([,.!?;:])/g, "$1").replace(/[ \t]{2,}/g, " ").trim();
  return joined.replace(/(^|[.!?]\s+|\n\s*)([a-z])/g, function(m, pre, ch){ return pre + ch.toUpperCase(); });
}

/* ================ FORMAT ENGINES (input parsers + report writers) ================ */
function decodeEntities(s){
```

```
  return String(s).replace(/&lt;/g, "<").replace(/&gt;/g, ">").replace(/&quot;/g, '"')
    .replace(/&apos;/g, "'").replace(/&#(\d+);/g, function(_, n){ return String.fromCharCode(+n); })
    .replace(/&#x([0-9a-fA-F]+);/g, function(_, n){ return String.fromCharCode(parseInt(n, 16)); })
    .replace(/&amp;/g, "&");
}
function escXml(s){ return String(s).replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;"); }
/* ---- ZIP reading (docx / odt / pdf containers) ---- */
function zipEntryMap(u8, dv){
  var eocd = -1;
  for (var i = u8.length - 22; i >= Math.max(0, u8.length - 66000); i--)
    if (dv.getUint32(i, true) === 0x06054b50){ eocd = i; break; }
  if (eocd < 0) return null;
  var count = dv.getUint16(eocd + 10, true), off = dv.getUint32(eocd + 16, true), entries = {};
  for (var n = 0; n < count; n++){
    if (dv.getUint32(off, true) !== 0x02014b50) break;
    var nlen = dv.getUint16(off + 28, true), elen = dv.getUint16(off + 30, true), clen = dv.getUint16(off + 32,
true);
    var name = new TextDecoder().decode(u8.subarray(off + 46, off + 46 + nlen));
    entries[name] = { method: dv.getUint16(off + 10, true), csize: dv.getUint32(off + 20, true), lhofs:
dv.getUint32(off + 42, true) };
    off += 46 + nlen + elen + clen;
  }
  return entries;
}
async function zipEntryBytes(u8, dv, entries, name){
  var e = entries[name]; if (!e) return null;
  var lo = e.lhofs, nlen = dv.getUint16(lo + 26, true), elen = dv.getUint16(lo + 28, true);
  var comp = u8.subarray(lo + 30 + nlen + elen, lo + 30 + nlen + elen + e.csize);
  if (e.method === 0) return comp.slice();
  if (e.method === 8){
    if (typeof DecompressionStream === "undefined") throw new Error("This browser cannot unpack Office files.
Please use a current Chrome, Edge, Firefox or Safari — or save as .txt.");
    var stream = new Blob([comp]).stream().pipeThrough(new DecompressionStream("deflate-raw"));
    return new Uint8Array(await new Response(stream).arrayBuffer());
  }
  throw new Error("Unsupported compression in the file.");
}
/* ---- DOCX text ---- */
function parseDocxText(xml){
  var paras = xml.split(/<w:p[\s>]/).slice(1).map(function(chunk){
    var end = chunk.indexOf("</w:p>");
    var body = end >= 0 ? chunk.slice(0, end) : chunk;
    body = body.replace(/<w:tab[^>]*\/>/g, " ").replace(/<w:br[^>]*\/>/g, "\n");
    var runs = body.match(/<w:t[^>]*>([\s\S]*?)<\/w:t>/g) || [];
    return runs.map(function(r){ return decodeEntities(r.replace(/<[^>]+>/g, "")); }).join("");
  });
  return paras.join("\n\n").replace(/[ \t]+\n/g, "\n").trim();
}
/* ---- ODT text ---- */
function parseOdtText(xml){
  var withBreaks = xml.replace(/<text:p[^>]*>/g, "\n\n").replace(/<text:line-break\/>/g, "\n").replace(/
<text:s[^>]*\/>/g, " ");
  return decodeEntities(withBreaks.replace(/<[^>]+>/g, "")).replace(/[ \t]{2,}/g, " ").trim();
}
/* ---- RTF text ---- */
var RTF_SKIP_DEST = { fonttbl: 1, colortbl: 1, stylesheet: 1, info: 1, pict: 1, object: 1, header: 1, footer: 1,
  footnote: 1, annotation: 1, fldinst: 1, listtable: 1, listoverridetable: 1, rsidtbl: 1, generator: 1,
  themedata: 1, colorschemedata: 1, datastore: 1, latentstyles: 1, xmlnstbl: 1, datafield: 1 };
var CP1252_HIGH = { 0x91: "‘", 0x92: "’", 0x93: "“", 0x94: "”", 0x95: "•", 0x96: "–", 0x97: "—", 0x85: "…",
0xA0: " " };
function parseRtf(rtf){
  var out = [], i = 0, n = rtf.length, depth = 0, skipDepth = -1;
  function isSkipping(){ return skipDepth >= 0; }
  while (i < n){
    var ch = rtf[i];
    if (ch === "{"){
      // peek: group begins with \* or a destination keyword?
      var m = rtf.slice(i + 1, i + 40).match(/^\s*(\\\*)|^\s*\\([a-z]+)/);
      depth++;
      if (!isSkipping() && m){
        var kw = m[2] || (m[1] ? "*" : "");
        if (kw === "*" || RTF_SKIP_DEST[kw]) skipDepth = depth;
      }
      i++; continue;
    }
    if (ch === "}"){
      if (skipDepth === depth) skipDepth = -1;
```

```
      depth--; i++; continue;
    }
    if (ch === "\\"){
      var rest = rtf.slice(i + 1);
      var cm = rest.match(/^([a-zA-Z]+)(-?\d+)? ?/);
      if (cm){
        var word = cm[1], arg = cm[2] !== undefined ? parseInt(cm[2], 10) : null;
        if (!isSkipping()){
          if (word === "par" || word === "line") out.push("\n");
          else if (word === "tab") out.push("\t");
          else if (word === "u" && arg !== null){ out.push(String.fromCharCode(arg < 0 ? arg + 65536 : arg)); i
+= 1 + cm[0].length; if (rtf[i] === "?") i++; continue; }
          else if (word === "emdash") out.push("—");
          else if (word === "endash") out.push("–");
          else if (word === "bullet") out.push("•");
          else if (word === "lquote") out.push("‘");
          else if (word === "rquote") out.push("’");
          else if (word === "ldblquote") out.push("“");
          else if (word === "rdblquote") out.push("”");
        }
        i += 1 + cm[0].length; continue;
      }
      var hx = rest.match(/^'([0-9a-fA-F]{2})/);
      if (hx){
        if (!isSkipping()){
          var code = parseInt(hx[1], 16);
          out.push(CP1252_HIGH[code] || String.fromCharCode(code));
        }
        i += 4; continue;
      }
      var sym = rest[0];
      if (!isSkipping()){
        if (sym === "~") out.push(" ");
        else if (sym === "_") out.push("-");
        else if (sym === "\\" || sym === "{" || sym === "}") out.push(sym);
      }
      i += 2; continue;
    }
    if (!isSkipping() && ch !== "\r" && ch !== "\n") out.push(ch);
    i++;
  }
  return out.join("").replace(/\n{3,}/g, "\n\n").replace(/[ \t]+\n/g, "\n").trim();
}
/* ---- HTML text ---- */
function stripHtml(html){
  var s = String(html)
    .replace(/<script[\s\S]*?<\/script>/gi, " ").replace(/<style[\s\S]*?<\/style>/gi, " ")
    .replace(/<!--[\s\S]*?-->/g, " ")
    .replace(/<\/(p|div|li|tr|h[1-6]|blockquote)>/gi, "\n\n")
    .replace(/<br\s*\/?>/gi, "\n").replace(/<li[^>]*>/gi, "\n• ")
    .replace(/<[^>]+>/g, "");
  return decodeEntities(s.replace(/ /g, " ")).replace(/[ \t]{2,}/g, " ").replace(/\n{3,}/g, "\n\n").trim();
}
/* ---- PDF text (best effort) ---- */
function pdfUnescape(s){
  return s.replace(/\\([0-7]{1,3})/g, function(_, o){ return String.fromCharCode(parseInt(o, 8)); })
    .replace(/\\n/g, "\n").replace(/\\r/g, "").replace(/\\t/g, "\t")
    .replace(/\\\(/g, "(").replace(/\\\)/g, ")").replace(/\\\\/g, "\\");
}
async function inflateStream(bytes){
  if (typeof DecompressionStream === "undefined") return null;
  for (var fmt of ["deflate", "deflate-raw"]){
    try {
      var stream = new Blob([bytes]).stream().pipeThrough(new DecompressionStream(fmt));
      return new Uint8Array(await new Response(stream).arrayBuffer());
    } catch (e) { /* try next */ }
  }
  return null;
}
function ascii85Decode(str){
  var data = String(str).replace(/\s+/g, "");
  if (data.indexOf("<~") === 0) data = data.slice(2);
  if (data.slice(-2) === "~>") data = data.slice(0, -2);
  var out = [], group = [];
  function emit(g, count){
    var num = 0;
    for (var k = 0; k < 5; k++) num = num * 85 + g[k];
```

```
    for (var b = 0; b < count; b++) out.push(Math.floor(num / Math.pow(256, 3 - b)) % 256);
  }
  for (var i = 0; i < data.length; i++){
    var ch = data[i];
    if (ch === "z" && group.length === 0){ out.push(0, 0, 0, 0); continue; }
    var v = ch.charCodeAt(0) - 33;
    if (v < 0 || v > 84) continue;
    group.push(v);
    if (group.length === 5){ emit(group, 4); group = []; }
  }
  if (group.length){ var rem = group.length; while (group.length < 5) group.push(84); emit(group, rem - 1); }
  return new Uint8Array(out);
}
async function parsePdfText(buf){
  var u8 = new Uint8Array(buf);
  var latin = new TextDecoder("latin1").decode(u8);
  var chunks = [], re = /\bstream\r?\n/g, m;
  while ((m = re.exec(latin)) !== null){
    var start = m.index + m[0].length;
    var end = latin.indexOf("endstream", start);
    if (end < 0) continue;
    while (end > start && (u8[end - 1] === 10 || u8[end - 1] === 13)) end--;
    chunks.push(u8.subarray(start, end));
  }
  var texts = [];
  for (var ci = 0; ci < chunks.length; ci++){
    var raw = await inflateStream(chunks[ci]);
    if (!raw){
      var a85 = ascii85Decode(new TextDecoder("latin1").decode(chunks[ci]));
      if (a85 && a85.length) raw = (await inflateStream(a85)) || a85;
    }
    var content = raw ? new TextDecoder("latin1").decode(raw) : null;
    if (!content || content.indexOf("BT") < 0) continue;
    var out = [], tok = content.match(/\((?:\\.|[^\\()])*\)\s*(Tj|TJ|'|")|\[(?:\\.|[^\]])*\]\s*TJ|(T\*|Td|TD)\b/
g) || [];
    // simpler pass: walk operators in order
    out = [];
    var re2 = /\((?:\\.|[^\\()])*\)|\[(?:\\.|[^\]])*\]|[A-Za-z'"*]{1,3}/g, t, pending = [];
    while ((t = re2.exec(content)) !== null){
      var v = t[0];
      if (v[0] === "(") pending.push(pdfUnescape(v.slice(1, -1)));
      else if (v[0] === "["){
        var parts = v.match(/\((?:\\.|[^\\()])*\)|-?\d+\.?\d*/g) || [], line = "";
        parts.forEach(function(p){
          if (p[0] === "(") line += pdfUnescape(p.slice(1, -1));
          else if (parseFloat(p) < -120) line += " ";
        });
        pending.push(line);
      }
      else if (v === "Tj" || v === "'" || v === '"'){ if (pending.length) out.push(pending.join("")); pending =
[]; out.push("\n"); }
      else if (v === "TJ"){ if (pending.length) out.push(pending.join("")); pending = []; out.push("\n"); }
      else if (v === "T*" || v === "Td" || v === "TD") out.push("\n");
      else if (v === "ET") out.push("\n");
    }
    texts.push(out.join(""));
  }
  var text = texts.join("\n").replace(/[ \t]{2,}/g, " ").replace(/\n{3,}/g, "\n\n").trim();
  var wordCount = (text.match(/[A-Za-z]{2,}/g) || []).length;
  if (wordCount < 15) throw new Error("readable text could not be extracted from this PDF (it may be scanned
images or use embedded font encodings). Please copy the text and paste it instead.");
  return text;
}
/* ---- CRC32 + stored-ZIP writer (for .docx export) ---- */
var CRC_TABLE = null;
function crc32(bytes){
  if (!CRC_TABLE){
    CRC_TABLE = [];
    for (var n = 0; n < 256; n++){ var c = n; for (var k = 0; k < 8; k++) c = c & 1 ? 0xEDB88320 ^ (c >>> 1) : c
>>> 1; CRC_TABLE[n] = c >>> 0; }
  }
  var crc = 0xFFFFFFFF;
  for (var i = 0; i < bytes.length; i++) crc = CRC_TABLE[(crc ^ bytes[i]) & 0xFF] ^ (crc >>> 8);
  return (crc ^ 0xFFFFFFFF) >>> 0;
}
function utf8Bytes(s){ return new TextEncoder().encode(s); }
function zipStore(files){ // files: [{name, bytes(Uint8Array)}]
```

```
  function u16(v){ return [v & 0xFF, (v >> 8) & 0xFF]; }
  function u32(v){ return [v & 0xFF, (v >> 8) & 0xFF, (v >> 16) & 0xFF, (v >> 24) & 0xFF]; }
  var out = [], central = [], offset = 0;
  files.forEach(function(f){
    var nameB = utf8Bytes(f.name), crc = crc32(f.bytes), size = f.bytes.length;
    var local = [].concat(u32(0x04034b50), u16(20), u16(0x0800), u16(0), u16(0), u16(0x21), u32(crc), u32(size),
u32(size), u16(nameB.length), u16(0));
    out.push(new Uint8Array(local), nameB, f.bytes);
    var cen = [].concat(u32(0x02014b50), u16(20), u16(20), u16(0x0800), u16(0), u16(0), u16(0x21), u32(crc),
u32(size), u32(size),
      u16(nameB.length), u16(0), u16(0), u16(0), u16(0), u32(0), u32(offset));
    central.push(new Uint8Array(cen), nameB);
    offset += local.length + nameB.length + size;
  });
  var cenSize = central.reduce(function(a, b){ return a + b.length; }, 0);
  var end = new Uint8Array([].concat(u32(0x06054b50), u16(0), u16(0), u16(files.length), u16(files.length),
u32(cenSize), u32(offset), u16(0)));
  var total = offset + cenSize + end.length, zip = new Uint8Array(total), pos = 0;
  out.concat(central, [end]).forEach(function(part){ zip.set(part, pos); pos += part.length; });
  return zip;
}
/* ---- plain text wrapping + simple text-PDF writer ---- */
function wrapText(text, width){
  var lines = [];
  String(text).split("\n").forEach(function(para){
    var words = para.split(/\s+/).filter(Boolean), cur = "";
    if (!words.length){ lines.push(""); return; }
    words.forEach(function(w){
      if ((cur + " " + w).trim().length > width && cur){ lines.push(cur); cur = w; }
      else cur = (cur + " " + w).trim();
    });
    if (cur) lines.push(cur);
  });
  return lines;
}
function pdfSanitize(s){
  return String(s).replace(/[‘’]/g, "'").replace(/[“”]/g, '"').replace(/[—–]/g, "-")
    .replace(/•/g, "-").replace(/[^\x20-\x7E\xA0-\xFF]/g, "?");
}
function buildTextPdf(title, bodyText){
  var lines = wrapText(pdfSanitize(bodyText), 92);
  var perPage = 46, pages = [];
  for (var i = 0; i < lines.length; i += perPage) pages.push(lines.slice(i, i + perPage));
  if (!pages.length) pages.push([]);
  var objects = [], content = [];
  // objects: 1 catalog, 2 pages, 3 font, then per page: page obj + content obj
  var pageObjNums = pages.map(function(_, pi){ return 4 + pi * 2; });
  objects[1] = "<< /Type /Catalog /Pages 2 0 R >>";
  objects[2] = "<< /Type /Pages /Kids [" + pageObjNums.map(function(n){ return n + " 0 R"; }).join(" ") + "] /
Count " + pages.length + " >>";
  objects[3] = "<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica /Encoding /WinAnsiEncoding >>";
  pages.forEach(function(pageLines, pi){
    var pageNum = 4 + pi * 2, contentNum = pageNum + 1;
    objects[pageNum] = "<< /Type /Page /Parent 2 0 R /MediaBox [0 0 595 842] /Resources << /Font << /F1 3 0 R >>
>> /Contents " + contentNum + " 0 R >>";
    var ops = ["BT /F1 10.5 Tf 14 TL 48 800 Td"];
    if (pi === 0){
      ops.push("/F1 15 Tf (" + pdfEscape(title) + ") Tj /F1 10.5 Tf T* T*");
    }
    pageLines.forEach(function(l){ ops.push("(" + pdfEscape(l) + ") Tj T*"); });
    ops.push("ET");
    var stream = ops.join("\n");
    objects[contentNum] = "<< /Length " + stream.length + " >>\nstream\n" + stream + "\nendstream";
  });
  var pdf = "%PDF-1.4\n", offsets = [0];
  var maxObj = 3 + pages.length * 2;
  for (var o = 1; o <= maxObj; o++){
    offsets[o] = pdf.length;
    pdf += o + " 0 obj\n" + objects[o] + "\nendobj\n";
  }
  var xrefPos = pdf.length;
  pdf += "xref\n0 " + (maxObj + 1) + "\n0000000000 65535 f \n";
  for (var o2 = 1; o2 <= maxObj; o2++) pdf += ("0000000000" + offsets[o2]).slice(-10) + " 00000 n \n";
  pdf += "trailer\n<< /Size " + (maxObj + 1) + " /Root 1 0 R >>\nstartxref\n" + xrefPos + "\n%%EOF";
  var bytes = new Uint8Array(pdf.length);
  for (var b = 0; b < pdf.length; b++) bytes[b] = pdf.charCodeAt(b) & 0xFF;
  return bytes;
```

```
}
function pdfEscape(s){ return pdfSanitize(s).replace(/\\/g, "\\\\").replace(/\(/g, "\\(").replace(/\)/g, "\
\)"); }
```